

Lecture 02

In this Session we only will explore Python core primitive types, we will look at the literals of these types and investigate some of the operations that can be performed on them.

At in this chapter we will do a cursory exploration of the more complex types (such as tuples, strings, lists, sets and dict), however, but we will covered them in more thoroughly in subsequent notebooks. At this point you are only require to recognise the literals of these types and to be able to print them.

Introduction to Python Data Types

Everything in Python is an object with its own type.

You may use the following url to [visualize](#) your code

Contents

1. [None](#)
2. [Numbers](#)
 - Integer (`int`)
 - Int literals
 - Operations on Ints
 - More Operation on Ints (Bit fiddling)
 - Float (`float`)
 - Complex (`complex`) (info only)
 - Boolean (`bool`)
 - Arithmetic operation between numerics
 - Arithmetic operation between numbers and non-numerics
 - Fraction (info only) Available via an explicit import
 - Decimal (info only) Available via an explicit import
3. [Sequences](#)
 - Tuples (`tuple`) - Immutable, ordered and indexable
 - String (`str`) - Sequence of unicode characters
 - List (`list`) - Mutable, ordered and indexable. (heavily used)
4. [Sets](#)
 - Set (`set`) Mutable, unordered and not indexable. No duplicates.
5. [Mappings](#)
 - Dictionary (`dict`) - Mutable and ordered. (heavily used)
6. [Enum](#) (info only) Available via an explicit import
7. Checking Data Types
8. [Summary](#)

What are Data Types?

In programming, a data type is a classification that specifies:

- What kinds of value a variable can hold
- What operation can be preformed on the variable
 - arithmetic can only be preformed on numerics
 - str can be capitalized
 - a set is guaranteed to have all unique elements

You can infer the type of a literal by inspection:

- `str`: anything enclosed by single, double or triple quotation
- `int`: series of digits
- `float`: series of digits with a period
- `bool`: True or False
- `set`: items separated by commas and enclosed by a pair of curly braces
- `tuple`: items separated by commas and enclosed by a pair of parentheses
- `list`: items separated by commas and enclosed by a pair of square brackets
- `dict`: pairs separated by commas and enclosed by a pair of curly braces

You are reminded about the coding practices that should be adhered to for this course regarding file name, identifiers and also pasting a copy of the problem at the top of your python file.

1. None

Is used to define a null value, or no value at all. It is not the same as 0, `False`, or an empty string. Comparing `None` to anything except `None` will return `False`. Probably similar to the `c# null` value.

N.B. In this course you will use f-strings in your output. Placeholders or other printing styles will not be tolerated.

```
a = None
print(f'type of {a} => {type(a)}')
```



```
value = False
print(f'{a} == {value} => {a == value}')           #False
```



```
value = True
print(f'{a} == {value} => {a == value}')           #False
```



```
value = 0
print(f'{a} == {value} => {a == value}')           #False
```



```
value = ''
print(f'{a} == '{value}' => {a == value}')         #False
```

```

value = None
print(f'{a} == {value} => {a == value}')           #True
type of None => <class 'NoneType'>
None == False => False
None == True  => False
None == 0     => False
None == ''    => False
None == None  => True

```

2. Numbers

Numbers are immutable. Same as string in C#.

Why do we say that numbers are immutable?

When you perform an operation on a number, the result is computed and stored in a new location but accessed with the same variable. The value at the original location remains the same.

Python has four built-in number types and two other accessible via explicit imports:

1. [Integers](#)
2. [Floats](#)
3. [Complex numbers](#)
4. [Booleans](#)
5. [Fractions](#) via import fraction
6. [Decimals](#) via import decimal

2.1 Integers

This type is used to work with negative and positive whole numbers.

C# has:

- 8-bits such as sbyte and byte
- 16-bits such as short and ushort
- 32-bits such as int and uint
- 64-bits such as long and ulong

In Python there is only one size of integer.

Integers are limited only by the available memory of the computer.

2.1.1 Int literal

A literal is the actual value. So an int literal will be like 3 or -1

```

# Integer Literals
# Python supports decimal, binary, octal, and hexadecimal integer literals.

#decimal literal
a = 11
print(type(a))
print(a)

#binary literal, this starts with 0b or 0B
b = 0b1011
print(b)

#octal literal, this starts with 0o or 0O
c = 0o13
print(c)

#hexadecimal, this starts with 0x or 0X
d = 0xb
print(d)

#Like C#, python allows underscores to make
#number literals more readable
e = 1_100_000
print(e)
f = 0x_4_ad
print(f)

#Like C#, you may also use format specifier for a number literal
print(f'{e:,}')
<class 'int'>
11
11
11
11
1100000
1197
1,100,000

```

Bit fiddling

The `<<` and `>>` operators allow you to shift the bits left or right resulting in multiplying or dividing by powers of 2 i.e. 2, 4, 8, 16.

To better understand these operations, we will look at the actions on binary literals.

```

a = 13
factor = 1
c = a << factor
print(f'{a} << {factor} => {c}  ≡ [{bin(a)} << {factor} will give =>
{bin(c)}]')

factor = 2
c = a << factor

```

```

print(f'{a} << {factor} => {c}  ≡ [{bin(a)} << {factor} will give =>
{bin(c)}]')

factor = 3
c = a << factor
print(f'{a} << {factor} => {c} ≡ [{bin(a)} << {factor} will give =>
{bin(c)}]')

factor = 1
c = a >> factor
print(f'{a} >> {factor} => {c}  ≡ [{bin(a)} >> {factor} will give =>
{bin(c)}]')

factor = 2
c = a >> factor
print(f'{a} >> {factor} => {c}  ≡ [{bin(a)} >> {factor} will give =>
{bin(c)}]')

factor = 3
c = a >> factor
print(f'{a} >> {factor} => {c}  ≡ [{bin(a)} >> {factor} will give =>
{bin(c)}]')

b = 8
print(f'{a} &  {b} => {a & b}    ≡ [{bin(a)} & {bin(b)} >> {bin(a & b)}]')

print(f'{a} |  {b} => {a | b}    ≡ [{bin(a)} | {bin(b)} >> {bin(a | b)}]')

print(f'{a} ^  {b} => {a ^ b}    ≡ [{bin(a)} ^ {bin(b)} >> {bin(a ^ b)}]')
#a = 8
print("flips all the bits (note the results in two's complement)")
print(f'    ~{a} => {~a} ≡ [~{bin(a)} => {bin(~a)}]')
13 << 1 => 26  ≡ [0b1101 << 1 will give => 0b11010]
13 << 2 => 52  ≡ [0b1101 << 2 will give => 0b110100]
13 << 3 => 104 ≡ [0b1101 << 3 will give => 0b1101000]
13 >> 1 => 6   ≡ [0b1101 >> 1 will give => 0b110]
13 >> 2 => 3   ≡ [0b1101 >> 2 will give => 0b11]
13 >> 3 => 1   ≡ [0b1101 >> 3 will give => 0b1]
13 &  8 => 8   ≡ [0b1101 & 0b1000 >> 0b1000]
13 |  8 => 13  ≡ [0b1101 | 0b1000 >> 0b1101]
13 ^  8 => 5   ≡ [0b1101 ^ 0b1000 >> 0b101]
flips all the bits (note the results in two's complement)
~13 => -14 ≡ [~0b1101 => -0b1110]
# You can also use the `int()` constructor to convert a string to an integer.
# The string must represent a valid integer literal.
a = int('123')
print(f'{a} is of type {type(a)}')

# The `int()` constructor can also take a second argument to specify the base
of the number.
a = int('1011', 2)  # binary

```

```

print(f'{a} is of type {type(a)}') # 11 in decimal

a = int('13', 8) # octal
print(f'{a} is of type {type(a)}') # 11 in decimal
123 is of type <class 'int'>
11 is of type <class 'int'>
11 is of type <class 'int'>
#demonstrate immutability of numbers
micheal = 10
print(id(micheal))
micheal = micheal + 1 #new variable is created with the same name
print(micheal)
print(id(micheal))
140704902444232
11
140704902444264

```

2.1.2 Obtaining an int using the int() function

The `int()` function takes a string, a real number or a bytes-like object and returns an int

```

numberOfSiblings = input('How many siblings do you have? ')
hope = int(numberOfSiblings) # converts string to int
                                # if the user enters a number with a
                                # the previous statement will crash the
                                # Unless. specifically asked, mitigating
                                # is not required as part of your work
fraction
program

user input errors

print(f'I have {hope + 2} siblings')

#obtaining an int from a float and a boolean
print(int(3.14))
print(int(True))

# the int() does not work with a complex number
# print(int(3+2j))
I have 6 siblings
3
1

```

2.2 Float

The `float` type is used to store numbers with a fractional part i.e. a number that has a decimal point.

Any arithmetic operation between two floats will result in a float. Python's float has unlimited precision.

C# has

- 32-bits float
- 64-bits double

Float literal

A float literal is a number with a fraction part.

Python only support decimal literal for float, binary, octal and hexadecimal are not permitted.

```
#some float literals
a = 3.1428
print(type(a))
print(a)          #3.12428
b = 10.
c = .001

#using exponent notation
d = 2.3e10
print(d)

e = 5.3e-10
print(e)

pi = 3.1415
radius = 1
print(f'Value of pi is {pi}')
print(f'Type of pi is {type(pi)}')          #the type() function returns the
data type of the argument
print(f'A circle of radius {radius} will have an area of {pi * radius ** 2}')
#a more clean way
print(f'A circle of radius {radius:.2f} will have an area of {pi * radius **
2:.2f}')
<class 'float'>
3.1428
23000000000.0
5.3e-10
Value of pi is 3.1415
Type of pi is <class 'float'>
A circle of radius 1 will have an area of 3.1415
A circle of radius 1.00 will have an area of 3.14
pi = 3.1415
radius = 1

print(f'Value of pi is {pi}')
print(f'Type of pi is {type(pi)}')          #the type() function returns the
data type of the argument
print(f'A circle of radius {radius} will have an area of {pi * radius ** 2}')
#a more clean way
print(f'A circle of radius {radius:.2f} will have an area of {pi * radius **
2:.2f}')

#Arithmetic works the same way for ints and floats
```

```

a = 1.02
b = 6.13
c = f'{a} + {b}'
print(f'{c} = {eval(c)}')           #the eval() function evaluates an
expression

c = f'{a} - {b}'
print(f'{c} = {eval(c)}')

c = f'{a} * {b}'
print(f'{c} = {eval(c)}')

c = f'{a} / {b}'
print(f'{c} = {eval(c)}')

c = f'{b} // {a}'
print(f'{c} = {eval(c)}')

c = f'{a} ** {b}'
print(f'{c} = {eval(c)}')

c = f'{a} % {b}'
print(f'{c} = {eval(c)}')
Value of pi is 3.1415
Type of pi is <class 'float'>
A circle of radius 1 will have an area of 3.1415
A circle of radius 1.00 will have an area of 3.14
1.02 + 6.13 = 7.15
1.02 - 6.13 = -5.1099999999999999
1.02 * 6.13 = 6.2526
1.02 / 6.13 = 0.16639477977161501
6.13 // 1.02 = 6.0
1.02 ** 6.13 = 1.1290652808457022
1.02 % 6.13 = 1.02

```

2.2.2 Obtaining a float using the float() function

The `float()` function takes a string, int or bool and returns a float

```

print(float('inf'))           #infinity
print(float('NaN'))           #non-a-number
print(float(False))           #boolean
print(float(10))              #integer
print(float('3.141592'))     #float

#complex number, this will crash the program
# print(float(3+2j))
inf
nan
0.0
10.0
3.141592
# Demonstrating type conversion and user input

```



```
kg_weight = float(input('How heavy is your backpack (in kg? '))
lb_weight = kg_weight * 2.20462
print(f'Your backpack weighs {lb_weight:.2f} lbs')
Your backpack weighs 11.46 lbs
```

2.3 Complex number

This is similar to the mathematical concept of complex number.

There is a real and an imaginary part. The imaginary part is mandatory!

```
#a complex MUST have atleast an imaginary part
a = 2j
print(type(a))
print(a)

b = 2+ 3.14j
print(b)

c = 2.178 - 4j
print(c)

#you may have just an imaginary component
d = 2.15e100j
print(d)

e = 1.25e-3j
print(e)
<class 'complex'>
2j
(2+3.14j)
(2.178-4j)
2.15e+100j
0.00125j
```

Arithmetic operations

Any arithmetic operations (+, -, *, /, //, **, %) between any two numeric types is supported.

+, -, * and / behaves like what you would expect in normal arithmetic operations.

// Floor Division - This operator does a division and discards the fractional part (if there is any).

** Power Operator - This operator raises the first operand to the power of the second operand.

% Modulus Operator - It does a division, but instead of returning the quotient, it returns the remainder.

ints, floats, bool and complex

Any arithmetic operation (except normal division) between two ints will result in an int

```
#arithmetic operators
```

```

# addition, subtraction, multiplication, division, exponentiation, modulus
# floor division
a = '1 + 3'
print(f' {a} => {eval(a)}')

a = '1 - 3'
print(f' {a} => {eval(a)}')

a = '1 * 3'
print(f' {a} => {eval(a)}')

# this is floor division, the result is rounded down to the nearest integer
a = '5 // 3'
print(f'{a} => {eval(a)}')

# this is normal division, the result is a float
a = '1 / 3'
print(f' {a} => {eval(a)}')

# exponentiation
a = '2 ** 3'
print(f'{a} => {eval(a)}')

# modulus operator, gives the remainder
a = '10 % 3'
print(f'{a} => {eval(a)}')
1 + 3 => 4
1 - 3 => -2
1 * 3 => 3
5 // 3 => 1
1 / 3 => 0.3333333333333333
2 ** 3 => 8
10 % 3 => 1
# any arithmetic operation between an int and a bool will result in an int
a = '1 + True' # True is 1
print(f'{a} => {eval(a)}')

a = '1 - False' # False is 0
print(f'{a} => {eval(a)}')

a = '3 * False' # True is 1
print(f'{a} => {eval(a)}')

a = '3 // True'
print(f'{a} => {eval(a)}')
1 + True => 2
1 - False => 1
3 * False => 0
3 // True => 3
# any arithmetic operation between two bools in an int
a = 'True + False'
print(f'{a} => {eval(a)}')

```

```

a = 'True - False'
print(f'{a} => {eval(a)}')

a = 'True * False'
print(f'{a} => {eval(a)}')

a = 'False // True'
print(f'{a} => {eval(a)}')
True + False => 1
True - False => 1
True * False => 0
False // True => 0
# Any arithmetic operation between two floats will result in a float
a = '1.0 + 3.0'
print(f'{a} => {eval(a)}')

a = '1.0 - 3.0'
print(f'{a} => {eval(a)}')
1.0 + 3.0 => 4.0
1.0 - 3.0 => -2.0
# Any arithmetic operation between a float and an int will result in a float
a = '1 + 3.0'
print(f'{a} => {eval(a)}')

a = '1 - 3.0'
print(f'{a} => {eval(a)}')
1 + 3.0 => 4.0
1 - 3.0 => -2.0
# Any arithmetic operation between a float and a bool will result in a float
a = '1.0 + True' # True is 1
print(f'{a} => {eval(a)}')

a = '5.0 ** False'
print(f'{a} => {eval(a)}')
1.0 + True => 2.0
5.0 ** False => 1.0
# Any arithmetic operation between two complex will result in a complex
a = 1j + 3j
print(a)
a = (1+1j) * (1-1j)
print(a)
a = (1+1j) / (1-1j)
print(a)
4j
(2+0j)
1j
# Any arithmetic operation between a complex and an int will result in a complex
a = 1 + 3j
print(a)

# Any arithmetic operation between a complex and a float will result in a complex

```

```

a = 1.0 + 3j
print(a)

# Any arithmetic operation between a complex and a bool will result in a
complex
(1+3j)
(1+3j)
a = 10 / 2          #division always give a float
print(f'{a} -> {type(a)}')

a = 10 // 3         #integer division
print(f'{a} -> {type(a)}')

a = 10 * 2j         #multiplication
print(type(a))

a = '11 ** 2'       #this is the exponentiation operator
print(f'{a}      => {eval(a)}')

a = '25 ** 0.5'     #square root
print(f'{a}      => {eval(a)}')

a = '64 ** (1/3)'   #cube root
print(f'{a}      => {eval(a)}')

a = '81 ** 0.25'    #fourth root
print(f'{a}      => {eval(a)}')

a = '25 % 3'        #modulus operator
print(f'{a}      => {eval(a)}')

a = '6.13 % 3'      #is not 0.13 because of floating point representation
print(f'{a}      => {eval(a)}')

a = '25 % 4.1'
print(f'{a}      => {eval(a)}')
5.0 -> <class 'float'>
3 -> <class 'int'>
<class 'complex'>
11 ** 2      => 121
25 ** 0.5    => 5.0
64 ** (1/3)  => 3.9999999999999996
81 ** 0.25   => 3.0
25 % 3       => 1
6.13 % 3     => 0.12999999999999999
25 % 4.1     => 0.400000000000000213

```

Table summarizing arithmetic operation between the various numerics

	int	bool	float	complex
int	int	int	float	complex
bool	int	int	float	complex
float	float	float	float	complex
complex	complex	float	complex	complex

2.4 Booleans

There are only two possible literal value: **True** or **False**

However any python object can be tested for truth value. All of the following will be considered False:

- None
- False
- Any zero numeric e.g. 0, 0.0, 0j
- Any empty sequence e.g. "", {}, []

Everything else will be considered **True**

The logical operators `and`, `or` and `not` results in bool value

The arithmetic operators results in numerical values. (True and False are treated as 1 and 0 respectively)

How would read in a `False` value from the user?

```
a = None
print(f'{a}    => {type(a)}')

print(f'{a}    => {bool(a)}')           #None literal

a = 0
print(f'{a}    => {bool(a)}')           #integer 0

a = 1
print(f'{a}    => {bool(a)}')           #integer non-zero

a = 0.
print(f'{a}    => {bool(a)}')           #float 0.0

a = 3.14
print(f'{a}    => {bool(a)}')           #float non-zero
```

```

a = ''
print(f'{a}      => {bool(a)}')      #empty string

a = 'Hello'
print(f'{a}  => {bool(a)}')          #non-empty string

a = []
print(f'{a}      => {bool(a)}')      #empty list

a = [2, 3]
print(f'{a} => {bool(a)}')          #non-empty list

a = {}
print(f'{a}      => {bool(a)}')      #empty dictionary

a = {1: 'narendra', 2: 'pershad'}    #non-empty dictionary
print(f'{a} => {bool(a)}')

a = ()
print(f'{a}      => {bool(a)}')      #empty tuple

a = (4, 'comp216')
print(f'{a} => {bool(a)}')          #non-empty tuple

a = '10 + True'
print(f'{a} => {bool(eval(a))}')      #True because 10 + 1 is non-zero

a = '0 + False'
print(f'{a} => {bool(eval(a))}')      #False because 0 + 0 is zero
None    => <class 'NoneType'>
None    => False
0        => False
1        => True
0.0      => False
3.14     => True
''       => True
Hello    => True
[]       => False
[2, 3]   => True
{}       => False
{1: 'narendra', 2: 'pershad'} => True
()       => False
(4, 'comp216') => True
10 + True => True
0 + False => False
true = True
false = False
print(true)
print(type(true))

print('\nSome logic operations')
c = f'{true} and {false}'
print(f'{c} => {eval(c)}')

```

```

c = f'{true} or {false}'
print(f'{c} => {eval(c)}')

print('\nSome arithmetic operations')
pi = 3.1415
c = f'{true} * {pi}'
print(f'{c} => {eval(c)}')

c = f'{false} * {pi}'
print(f'{c} => {eval(c)}')

c = f'{true} + {pi}'
print(f'{c} => {eval(c)}')

c = f'{false} + {pi}'
print(f'{c} => {eval(c)}')

c = f'{false} * {pi}'
print(f'{c} => {eval(c)}')
True
<class 'bool'>

```

Some logic operations
True and False => False
True or False => True

Some arithmetic operations
True * 3.1415 => 3.1415
False * 3.1415 => 0.0
True + 3.1415 => 4.1415000000000001
False + 3.1415 => 3.1415
False * 3.1415 => 0.0

2.5 Fractions

Fractions hold a rational numerator and denominator in their lowest forms.
This is not a built-in type because you have to do an import

```

from fractions import Fraction
a = Fraction(2, 6)    #this will be converted to the lowest form
print(type(a))
print(a)

b = Fraction(1, 12)
print(b)
c = 'a + b'
# print(f'{c} = {c.numerator} / {c.denominator}')
print(f'{a} + {b} => {eval(c)}')
<class 'fractions.Fraction'>
1/3
1/12

```

$1/3 + 1/12 \Rightarrow 5/12$

2.6 Decimal

Also this is not a built-in type because you have to do an import

```
from decimal import Decimal as D
a = D(3.14)
print(type(a))
print(a)

a = '3.14'
print(a)

b = D(0.1)
print(b)

c = D(3)
print(c)

print(D('1.4').as_integer_ratio())
<class 'decimal.Decimal'>
3.140000000000000124344978758017532527446746826171875
3.14
0.1000000000000000055511151231257827021181583404541015625
3
(7, 5)
```

3. Sequences

1. Immutable Sequences

- [Range](#)
- [String](#)
- [Tuple](#)

2. Mutable Sequences

- [List](#)
- [Set](#)

Range

The range object in Python is a built-in type used to represent an immutable sequence of numbers. It's commonly used in loops and other situations where you need to iterate over a series of values.

Syntax

- `range(stop)`
- `range(start, stop)`
- `range(start, stop, step)`

Where

start: the beginning of the sequence (default is 0).

stop: the end of the sequence (exclusive).

step: the difference between each number (default is 1).

We will see the full power of this object when we study loops in Chapter 4.

```
a = range(5)
print(a)
```

```
#to see the sequence, we can convert to a list
print(list(a))
```

```
a = range(-3, 2)
print(list(a))
```

```
a = range(2, -3, -1)
print(list(a))
```

```
a = range(100, 200, 20)
print(list(a))
range(0, 5)
[0, 1, 2, 3, 4]
[-3, -2, -1, 0, 1]
[2, 1, 0, -1, -2]
[100, 120, 140, 160, 180]
```

String

The **str** type is known as string in most other languages. It is used to store textual information. In the python language, strings are the most ubiquitous type and possibly the most flexible of all types. You can easily convert any data type to string and from other data types to strings.

Everything that is read from the console is a string and everything displayed on the console a string.

A string is delimited by single quotes, double quotes, triple single quotes or triple double quotes.

All strings are unicode string.

N.B. Unlike most modern languages, python does not have a char type

```
# personally I prefer single quotes
a = 'to be \t or not \n to be'
print(a)
```

```
print('In COMP100, the single quotes will be the standard way of specifying a
str literal!')
print('') #empty line
to be    or not
to be
```

In COMP100, the single quotes will be the standard way of specifying a str literal!

```
print('Narendra Pershad is\nEVIL!!')    #Embedding a newline in a str literal
Narendra Pershad is
EVIL!!
```

I use double quotes because it need the single quote in the string

```
print('The double quotes will be used only is special situations')
print("such as: Narendra's reign\n")    #Using a single quote in a str
literal
```

The double quotes will be used only is special situations
such as: Narendra's reign

you can achieve the same with single quotes

```
a = 'Narendra\'s toys'
```

```
print(a)
```

Narendra's toys

triple quotation is convenient for multiple line string

```
a = '''
```

```
    this is a multiple
    line string.
```

```
    And some more text
    '''
```

```
print(a)
```

```
    this is a multiple
    line string.
```

```
    And some more text
```

#notice the use of the line continuation symbol

#it is illegal to put ANY character after the slash

```
a = 'first line '\
```

```
'second line '\
```

```
'third line'
```

```
print(a)
```

first line second line third line

you can suppress the automatic line feed

```
print('You can suppress the automatic line feed', end='')    #the next output
continues on this this line
```

```
print(', by utilizing the second argument of the print function.')
```

You can suppress the automatic line feed, by utilizing the second argument of the print function.

you can specify lots of strings and it will work

```
print('you', 'can', 'specify', 'lots', 'of', 'strings', 'and it will work')
```

#the next output continues on this this line

you can specify lots of strings and it will work

You may specify non-keyboard character via hexcode

```
a = '\u00c4\u00e8'    #four digit hex value
```

```
print(a)
```

```
a = '\U000000c4\U000000e8' #eight digit hex value
print(a)
```

```
a = '\u263a'
print(a) # a smiley face
Äè
Äè
☺
```

Printing str variables

More on printing str literals

Although there are lots of ways of sending output to the screen, in this course we will use **only** f-strings for printing.

```
a = 'This str literal is stored in the variable "a"'
b = 'b '
c = b * 10
```

```
print(a)
print(b)
print(c)
```

```
print(f'This is another way of displaying the same variable: {a}')
This str literal is stored in the variable "a"
b
b b b b b b b b b b
This is another way of displaying the same variable: This str literal is stor
ed in the variable "a"
```

Obtaining a string

Use the `input()` function to obtain a string from the keyboard.

```
name = input('Enter the name of your instructor: ')
print(f'{name} is the best instructor at Centennial College')
Narendra is the best instructor at Centennial College
numberOfSiblings = input('How many siblings do you have? ')
print(f'I also have {numberOfSiblings} siblings')

#can you explain the output of the following line?
print(f'However, I was hoping for {numberOfSiblings * 2} siblings')
#we will fix this later
I also have 3 siblings
However, I was hoping for 33 siblings
```

Obtaining a string

Use the `str()` function to obtain a string from other datatypes.

```
#using the str() function to convert other data types to string
# pretty much everything can be converted to a string
```

```

print(str(1))          #integer
print(str(3.14))       #float
print(str(True))       #boolean
print(str(3+2j))       #complex
print(str([2, 3]))     #list
print(str((2, 3)))     #tuple
print(str({1: 'narendra', 2: 'pershad'})) #dictionary
print(str(None))       #None
print(str(Fraction(2, 6))) #fraction
print(str(D(3.14)))    #decimal
print(str(range(5)))   #range
print(str(b'hello'))   #bytes
print(str(bytearray(b'hello'))) #bytearray
1
3.14
True
(3+2j)
[2, 3]
(2, 3)
{1: 'narendra', 2: 'pershad'}
None
1/3
3.140000000000000124344978758017532527446746826171875
range(0, 5)
b'hello'
bytearray(b'hello')

```

Tuples

An immutable heterogenous collection of python objects

```

a = (0, 1, 2, 3, 4, 'five', 'six', ['nested', 'list'])
print(type(a))
print(a)
print(len(a))          #number of items in the sequence
<class 'tuple'>
(0, 1, 2, 3, 4, 'five', 'six', ['nested', 'list'])
8

```

List

A list is a mutable heterogenous collection that is enumerable

```

#manually specifying the items in a list
a = [0, 1, 2, 3, 4, 'five', 'six', ['nested', 'list']]
print(type(a))
print(a)
print(len(a))          #number of items in the sequence
print(a[5])            #the sixth item
<class 'list'>
[0, 1, 2, 3, 4, 'five', 'six', ['nested', 'list']]
8

```

five

4. Sets

Set

A set is a mutable heterogeneous collection of unique hashable python objects.

A set may not have a list or dictionary or a nested set as one of its member, because these types are not hashable.

Like all sequences, a set is iterable but it does not support indexing.

```
a = {0, 1, 2, 3, 4, 'five', 'six', 0, 3}
print(type(a))
print(a)                #duplicate items 0 and 3 are removed
print(len(a))           #number of items in the sequence
<class 'set'>
{0, 1, 2, 3, 4, 'six', 'five'}
7
```

5. Mappings

Dictionary

- A collection of key-value pairs.
 - Each key must be unique and immutable such as strings, number or tuples. A list may not be a key.
 - Like sets dictionary does not support indexing.
 - A dictionary is almost like a json object.
- Although a set is delimited by the same pair of curly braces, the items in a dictionary are key-value pair whilst the items in a set the items are just items!

```
a = {'red': 'Danger', 1: 'first', 'second': 2, ('a', 'set'): 'a tuple is my key'}

print(type(a))
print(f'{a=}')
print(len(a))           #number of items in the sequence
print(a['red'])          #the value pair of red
<class 'dict'>
a={'red': 'Danger', 1: 'first', 'second': 2, ('a', 'set'): 'a tuple is my key'}
4
Danger
```

5 Enum

Enums is not a built-in type, because you have to import it

```
from enum import Enum
```

```
class TrafficLight(Enum):  
    GREEN = 1  
    YELLOW = 2  
    RED = 4
```

```
print(TrafficLight.GREEN)  
print(TrafficLight.__members__)  
TrafficLight.GREEN  
{'GREEN': <TrafficLight.GREEN: 1>, 'YELLOW': <TrafficLight.YELLOW: 2>, 'RED':  
<TrafficLight.RED: 4>}
```

Summary

- Python has several built-in data types: numeric (int, float, complex, bool), sequences (string, tuple, list, range) sets (set) and mappings (dict).
 - string, tuple, range, list, set and dict will be covered in details in subsequent notebooks
 - complex (numbers) and enums will not be covered in this course
 - custom types will be the last topic in this course
- Each type has specific characteristics and operations that can be performed on them.
- We can convert between types when needed.
- Understanding data types is fundamental to writing effective Python code.

Quick Reference Table

Type	Example	Mutable?	Ordered?	Use Case
str	"hello"	No	Yes	Text data
int	42	No	N/A	Whole numbers
float	3.14	No	N/A	Decimal numbers
bool	True	No	N/A	True/False values
list	[1, 2, 3]	Yes	Yes	Changeable sequences
tuple	(1, 2, 3)	No	Yes	Fixed sequences

Type	Example	Mutable?	Ordered?	Use Case
dict	{"key": "value"}	Yes	No*	Key-value storage
set	{1, 2, 3}	Yes	No	Unique items
None	None	No	N/A	Absence of value